

Trabajo Práctico Integrador

**Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas**

Alumno: Federico Javier Doello

Índice

1. Introducción	3
2. Marco Teórico	3
2.1 Listas	3
2.2 Diccionarios	3
2.3 Funciones	3
2.4 Condicionales	4
2.5 Ordenamientos	4
2.6 Estadísticas básicas	4
2.7 Archivos CSV	4
3. Decisiones Técnicas y Arquitectura	4
4. Dificultades y Conclusiones	5
5. Bibliografía	5

1. Introducción

El presente trabajo consiste en desarrollar una aplicación en Python para gestionar información sobre países. El sistema lee datos desde un archivo CSV y ofrece un menú en consola con opciones de búsqueda, filtrado, ordenamiento y estadísticas. El objetivo es aplicar los conceptos aprendidos en Programación 1 en un proyecto integrador.

2. Marco Teórico

2.1 Listas

Las listas son una estructura de datos en Python que permite almacenar varios elementos en una sola variable. Los elementos tienen un orden, se pueden modificar y se accede a ellos por su posición (índice), siendo el primer elemento el índice 0. Se definen usando corchetes [].

En este proyecto, la lista principal almacena todos los países cargados desde el CSV. Cada elemento de esa lista es un diccionario que representa un país con sus cuatro datos.

2.2 Diccionarios

Los diccionarios son una estructura de datos que almacena pares clave:valor. A diferencia de las listas, no se accede a los elementos por índice sino por clave. Las claves deben ser únicas dentro de un mismo diccionario y se definen usando llaves { }.

En este proyecto, cada país se representa como un diccionario con las claves nombre, poblacion, superficie y continente, lo que permite acceder a cada dato de forma clara y directa.

2.3 Funciones

Las funciones son bloques de código que se definen una vez y se pueden reutilizar las veces que se necesite. Se crean con la palabra clave def. Permiten dividir el programa en partes más pequeñas, cada una con una responsabilidad específica.

En este proyecto se usaron funciones separadas para cada operación: cargar_paises(), agregar_pais(), buscar_pais(), filtrar_paises(), ordenar_paises() y mostrar_estadisticas(), entre otras.

2.4 Condicionales

Las estructuras condicionales (if, elif, else) permiten que el programa ejecute distintas acciones según si una condición es verdadera o falsa. Son fundamentales para validar datos y controlar el flujo del programa.

En este proyecto se usan para validar que los campos no estén vacíos, que los valores numéricos sean correctos y para manejar las distintas opciones del menú.

2.5 Ordenamientos

Ordenar una lista significa reorganizar sus elementos según un criterio. Python tiene la función `sorted()` que devuelve una nueva lista ordenada sin modificar la original. Se puede indicar el criterio con el parámetro `key` y el orden con `reverse=True` para descendente.

En este proyecto se usa `sorted()` con `key=lambda` para ordenar los países por nombre, población o superficie.

2.6 Estadísticas básicas

Python tiene funciones incorporadas como `max()`, `min()` y `sum()` que permiten obtener el valor máximo, mínimo y la suma de una colección de datos. Combinadas con expresiones `lambda` se pueden aplicar sobre listas de diccionarios.

En este proyecto se calculan el país con mayor y menor población, promedios de población y superficie, y la cantidad de países por continente.

2.7 Archivos CSV

Un archivo CSV (Comma Separated Values) es un archivo de texto donde cada fila representa un registro y los campos están separados por comas. Es un formato muy usado para intercambiar datos entre programas.

Python incluye el módulo `csv` en su biblioteca estándar. En este proyecto se usa `csv.DictReader` para leer el archivo y convertir cada fila en un diccionario automáticamente, y `csv.DictWriter` para guardar los cambios.

3. Decisiones Técnicas y Arquitectura

3.1 Estructura del proyecto

El proyecto se organiza en dos archivos:

- `main.py`: contiene toda la lógica del programa.
- `países.csv`: almacena el dataset base con 20 países.

3.2 Flujo del programa

- Al iniciar, el programa lee el archivo `países.csv` y carga los datos en una lista de diccionarios.
- Se muestra el menú principal con 8 opciones.
- Según la opción elegida se ejecuta la función correspondiente.
- Las operaciones que modifican datos guardan los cambios en el CSV automáticamente.
- El ciclo continúa hasta que el usuario elige la opción 0 (Salir).

3.3 Capturas de pantalla

6. Estadísticas
7. Mostrar todos los países
0. Salir
Opción: 4

Filtrar países
1. Por continente
2. Por rango de población
3. Por rango de superficie
Opción: 1
Continente: Asia

3 país/es encontrado(s):

Nombre	Población	Superficie (km²)	Continente
China	1,412,600,000	9,596,960	Asia
India	1,380,004,385	3,287,263	Asia
Japón	125,800,000	377,975	Asia

GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Mostrar todos los países
0. Salir
Opción: 1

C: > Users > feder > TPI PROGRAMACION > TPI PROGRAMACION.py > ...

```
1 import csv
2 import os
3
4 ARCHIVO_CSV = os.path.join(os.path.dirname(os.path.abspath(__file__)),
5                             "países.csv")
6
7 # LECTURA Y ESCRITURA DEL CSV
8
9
10 def cargar_países():
11     """Lee el archivo CSV y devuelve una lista de diccionarios."""
12     países = []
13     try:
14         with open(ARCHIVO_CSV, newline="", encoding="utf-8") as arc
15             lector = csv.DictReader(archivo)
16             for fila in lector:
17                 try:
18                     país = {
19                         "nombre": fila["nombre"].strip(),
20                         "poblacion": int(fila["poblacion"]),
21                         "superficie": int(fila["superficie"]),
22                         "continente": fila["continente"].strip()
23                     }
24                     países.append(país)
25                 except (ValueError, KeyError):
26                     print(f"Filas con formato incorrecto ignoradas: {
27                         }")
28             except FileNotFoundError:
29                 print(f"No se encontró el archivo '{ARCHIVO_CSV}'. Se inici
30                 return países
```

TERMI... + 2: Python: T... Lin 290, col 11 Espacios: 4 UTF-8 LF {} Python 3.14.3 Python 3.14 (64-bit) Go Live

PS C:\Users\feder> & C:/Python314/py

thon.exe "c:/Users/feder/TPI PROGRAMACION/TPI PROGRAMACION.py"

GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Mostrar todos los países
0. Salir
Opción:

C: > Users > feder > TPI PROGRAMACION > TPI PROGRAMACION.py > ...

```
1 import csv
2 import os
3
4 ARCHIVO_CSV = os.path.join(os.path.dirname(os.path.abspath(__file__)), "países.c
5
6
7 # LECTURA Y ESCRITURA DEL CSV
8
9
10 def cargar_países():
11     """Lee el archivo CSV y devuelve una lista de diccionarios."""
12     países = []
13     try:
14         with open(ARCHIVO_CSV, newline="", encoding="utf-8") as archivo:
15             lector = csv.DictReader(archivo)
16             for fila in lector:
17                 try:
18                     país = {
19                         "nombre": fila["nombre"].strip(),
20                         "poblacion": int(fila["poblacion"]),
21                         "superficie": int(fila["superficie"]),
22                         "continente": fila["continente"].strip()
23                     }
```

6. Estadísticas

7. Mostrar todos los países

0. Salir

Opción: 7

Nombre	Población	Superficie (km²)	Continente
Argentina	45,376,763	2,780,400	América
Brasil	213,993,437	8,515,767	América
Estados Unidos	331,000,000	9,833,517	América
México	126,014,024	1,964,375	América
Colombia	51,049,498	1,141,748	América
España	47,351,567	505,990	Europa
Francia	67,413,000	551,695	Europa
Alemania	83,149,300	357,022	Europa
Italia	60,317,116	301,340	Europa
Reino Unido	67,886,011	243,610	Europa
China	1,412,600,000	9,596,960	Asia
India	1,380,004,385	3,287,263	Asia
Japón	125,800,000	377,975	Asia
Rusia	144,104,080	17,098,242	Europa
Australia	25,499,884	7,692,024	Oceanía
Egipto	102,334,404	1,001,449	África
Nigeria	206,139,589	923,768	África
Sudáfrica	59,308,690	1,219,090	África
Marruecos	36,910,560	446,550	África
Canadá	38,005,238	9,984,670	América

```

3
4 ARCHIVO_CSV = os.path.join(os.path.dirname(os.path.abspath(__file__
5
6
7 # LECTURA Y ESCRITURA DEL CSV
8
9
10 def cargar_paises():
11     """Lee el archivo CSV y devuelve una lista de diccionarios."""
12     paises = []
13     try:
14         with open(ARCHIVO_CSV, newline="", encoding="utf-8") as archi
15             lector = csv.DictReader(archivo)
16             for fila in lector:
17                 try:
18                     pais = {
19                         "nombre": fila["nombre"].strip(),
20                         "poblacion": int(fila["poblacion"]),
21                         "superficie": int(fila["superficie"]),
22                         "continente": fila["continente"].strip()
23                     }
24                     paises.append(pais)
25                 except (ValueError, KeyError):
26                     print(f"Filas con formato incorrecto ignoradas: ")
27     except FileNotFoundError:
28         print(f"No se encontró el archivo '{ARCHIVO_CSV}'. Se inici
29     return paises
30

```

Opción: 6

Estadísticas

Población:

Mayor: China (1,412,600,000)

Menor: Australia (25,499,884)

Promedio: 231,212,877

Superficie:

Mayor: Rusia (17,098,242 km²)

Menor: Reino Unido (243,610 km²)

Promedio: 3,891,172 km²

Países por continente:

América: 6

Asia: 3

Europa: 6

Oceanía: 1

África: 4

GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Mostrar todos los países
0. Salir

Opción: 1

```

C:\Users> feder > TPI PROGRAMACION > TPI PROGRAMACION.py > ...
1 import csv
2 import os
3
4 ARCHIVO_CSV = os.path.join(os.path.dirname(os.path.abspath(__file__
5
6
7 # LECTURA Y ESCRITURA DEL CSV
8
9
10 def cargar_paises():
11     """Lee el archivo CSV y devuelve una lista de dicci
12     paises = []
13     try:
14         with open(ARCHIVO_CSV, newline="", encoding="utf
15             lector = csv.DictReader(archivo)
16             for fila in lector:
17                 try:
18                     pais = {
19                         "nombre": fila["nombre"].strip(
20                         "poblacion": int(fila["poblacion
21                         "superficie": int(fila["superfi
22                         "continente": fila["continente"]
23                     }
24                     paises.append(pais)
25                 except (ValueError, KeyError):
26                     print(f"Filas con formato incorrecto
27     except FileNotFoundError:
28         print(f"No se encontró el archivo '{ARCHIVO_CSV'

```

4. Dificultades y Conclusiones

4.1 Dificultades encontradas

- Al principio el programa no encontraba el archivo CSV porque lo buscaba en la carpeta desde donde se ejecutaba Python y no en la carpeta del proyecto. Se resolvió usando `os.path.dirname(os.path.abspath(__file__))` para que siempre busque el CSV en la misma carpeta que el archivo `.py`.
- Otro desafío fue validar las entradas del usuario para evitar que el programa se rompa al ingresar letras donde se esperaban números.

4.2 Conclusiones

Este trabajo me permitió poner en práctica los conceptos de Programación 1 en un proyecto real. Entendí mejor cómo funcionan las listas de diccionarios para organizar datos, la importancia de separar el código en funciones y cómo manejar archivos CSV para que los datos persistan entre ejecuciones.

5. Bibliografía

- Python Software Foundation. (2024). Estructuras de datos — documentación de Python 3. Recuperado de <https://docs.python.org/es/3/tutorial/datastructures.html>
- Python Software Foundation. (2024). csv — Lectura y escritura de archivos CSV — documentación de Python 3. Recuperado de <https://docs.python.org/es/3/library/csv.html>

Repositorio GitHub: <https://github.com/Feded2310/TPI-Programacion1>

Video demostrativo: <https://www.youtube.com/watch?v=JotQwvfoVkg&t=3s>